

WayBoxAI

Shipping document verification, from inbox to discrepancy report

An AI email triage system for shipping operations. It sorts an entire inbox into five categories, reads the Shipping Instruction and draft Bill of Lading attached to each document-comparison request, compares the seven mandated fields, and either reports the mismatch or escalates to a person with the reason and the evidence.

520/520

emails classified correctly

519/520

full verdicts exactly right

0

false alarms on 454 clean pairs

Project	WayBoxAI — AI email triage for logistics
Use case	Shipping document verification
Live prototype	https://wayboxai.vercel.app
Repository	https://github.com/TeyYongZhun/SleepyM4tcha-Hackathon
Document	Technical architecture, implementation, challenges and roadmap
Date	21 September 2026

1. Executive summary

A shipping operations team receives document-check requests, new SI requests, invoice queries, operational updates and spam in one inbox. Each document-check request requires a manual field-by-field comparison of a Shipping Instruction against a draft Bill of Lading. The work is repetitive, easy to get wrong, and a missed discrepancy becomes a correction, a delay, or a cost after the draft is finalised.

WayBoxAI automates the path from inbox to discrepancy report. It is a working, deployed web application: a three-pane mail client where every message arrives already sorted, already compared, and already explained.

Capability required	How WayBoxAI delivers it
Classify	Five-way classifier over the whole inbox: BL Comparison, SI Request, Invoice Query, General, Spam. Live counts on every tab. 520/520 on the supplied dataset.
Extract data	Reads <code>.txt</code> , <code>.pdf</code> , <code>.docx</code> and <code>.xlsx</code> attachments and identifies shipment fields, resolving synonymous labels ("Port of Loading" = "Load Port") with a trained label classifier.
Compare	Checks the seven mandated fields and shows SI and BL values side by side, highlighting only what differs. 45 of 46 mismatches found with exactly the right fields.
Ask for help	Four distinct escalation reasons rather than a guess or a silent failure, each carrying the evidence. All 20 review cases given the correct reason.

THE DESIGN PRINCIPLE

An honest "I cannot decide this" beats a confident wrong answer. A false alarm costs an operator's trust; a missed discrepancy costs a shipment. Every stage of both pipelines can stop and hand the case to a person with the reason attached — and on the 454 clean pairs in the dataset, the system raised **zero** false alarms.

2. Results

Scored against `ground_truth.json` as supplied by the organisers. Every figure below was reproduced by running the system end to end over all 520 emails immediately before this report was written.

What is measured	Correct	Total	Rate
Stage 1 — category	520	520	100%
Category, 5-fold cross-validation (500 training emails) macro-F1 1.000 in every fold; zero misclassified	500	500	100%
Stage 2 — verdict (status)	519	520	99.8%
Escalation reason (review_reason)	519	520	99.8%
Mismatched fields (defect_fields)	519	520	99.8%
All four exactly right	519	520	99.8%
Held-out edge cases <code>email_501 – email_520</code>	20	20	100%

What the dataset contains

Breakdown	Composition
Verdicts	454 OK · 46 MISMATCH · 20 NEEDS_REVIEW
Categories	220 BL Comparison · 125 SI Request · 75 Invoice Query · 60 General · 40 Spam
Escalation reasons	5 each of missing attachment, unreadable, wrong document type, missing value
Attachments	250 files — 192 .txt , 28 .pdf , 22 .xlsx , 8 .docx

Read across those rows: **45 of the 46 mismatches** are found with exactly the right fields, **all 20 review cases** get the right reason, and **all 454 clean pairs** are left alone.

On the category stage, the deterministic rule fired on 109 of the 500 training emails and the model decided the remaining 391, with **zero disagreements** between the two where both had an opinion — the rule encodes a real signal rather than patching model error.

The one miss, stated plainly

EMAIL_499

Ground truth expects a `MISMATCH` on gross weight. Its Bill of Lading PDF is truncated and raises `Unexpected EOF` when opened, so the system cannot read it and escalates as `NEEDS_REVIEW / unreadable` instead.

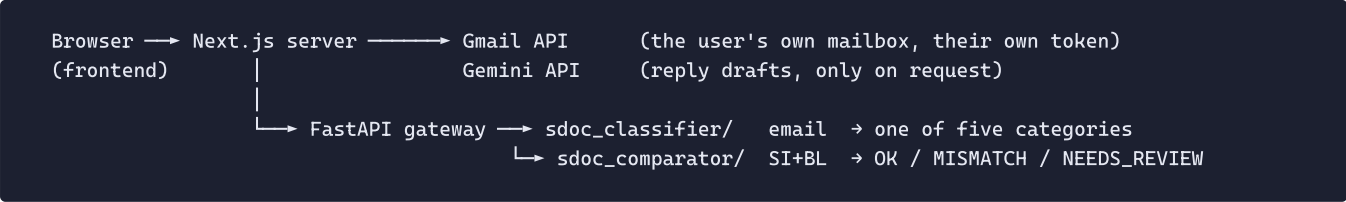
We consider this the correct engineering behaviour and have not special-cased it. Reporting a weight discrepancy from a file that could not be parsed would be a guess presented as a fact. The alternative — partial extraction from a corrupt PDF — trades one point of score for a class of silent, unverifiable errors in production.

How to reproduce

A script in the repository README runs the full 520-email evaluation in memory and prints the table above along with the single miss. The classifier half can be scored separately with `python -m src.evaluate` in `sdoc_classifier/`.

3. Technical architecture

The system is one repository deployed as two independent services, in two languages, connected by a single environment variable. The browser talks only to the web service; the web service talks to the model gateway.



Service	Responsibility	Runtime
Next.js server deployed on Vercel	Everything tied to a signed-in user: OAuth and token refresh, reading the mailbox, paging, attachment streaming, reply drafting, the local TL;DR summariser.	Node 20 · TypeScript
FastAPI gateway deployable to Railway	The model stages, and nothing else. Knows nothing about users. Give it a message and it returns a category; give it an SI and a BL and it returns a comparison.	Python 3.13 · scikit-learn

Why they are split. The models need Python and a long-lived process to stay loaded in memory; the web tier needs Node and scales per request. Separating them lets each scale on its own axis, and lets the gateway be replaced or retrained without redeploying the website.

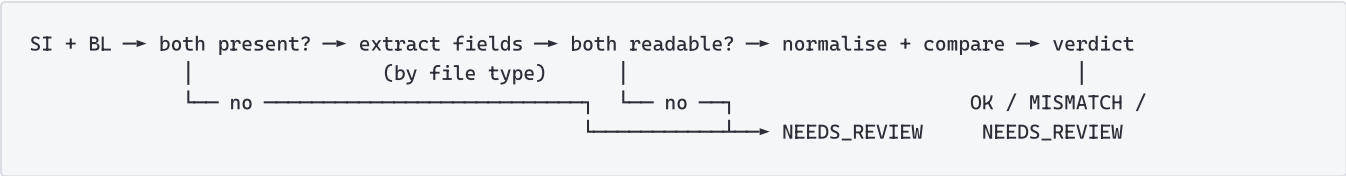
Graceful degradation is designed in. If the gateway is unreachable, the web service falls back to its own rule-based classification and comparison, and the inbox still renders. A model service being down must never empty a user's mailbox.

Pipeline 1 — categorising an email



- Parse** — sender, subject, body and attachment names into one record.
- Clean and flag** — normalise text, then compute engineered flags: internal sender domain, an SI+BL attachment pair, coded subject formats.
- Classify** — balanced logistic regression over TF-IDF returns a probability per category.
- Rule override** — a deterministic rule overrides the model where the signal is unambiguous (an SI and a BL attached together means BL Comparison).
- Confidence gate** — anything below 0.55 is reported as `GENERAL` rather than a confident wrong answer, with the model's original guess preserved in `model_category` so nothing is hidden.

Pipeline 2 — comparing an SI against a BL



- Presence check** — if only one of the pair arrived, stop. Comparing against nothing would report every field as missing, which reads as a catastrophic mismatch.

2. **Extract** — a small label classifier locates each field, dispatching on file type. This is where “Load Port” and “Port of Loading” are recognised as the same field.
3. **Readability check** — a corrupt file, a scan with no text layer, or a document that is not an SI or BL at all stops the run here with a specific reason.
4. **Normalise and compare** — deterministic rules reconcile spelling, spacing, casing and unit differences before two values are called a mismatch.
5. **Verdict** — OK , MISMATCH with the offending fields, or NEEDS_REVIEW with the reason a person is needed.

WHERE MACHINE LEARNING IS, AND IS NOT, USED

ML sorts the mail, and decides which field a raw document label refers to. The comparison itself is plain, readable code. Every verdict can be traced to a rule a reviewer can read, which is what makes the output defensible to an operations team that has to act on it.

4. Implementation details

Technology stack

Layer	Technology	Why
Frontend	Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS 4	Server components keep API keys and tokens off the client
Authentication	Auth.js v5, Google OAuth (<code>gmail.modify</code>)	Read the mailbox and clear unread marks; never send or delete
Mail source	Gmail API, paged 50 messages at a time	Stays inside the 15,000 units/minute quota on any inbox size
Classifier	scikit-learn, TF-IDF + engineered flags, balanced logistic regression	Fast, interpretable, and inspectable per prediction
Comparator	Trained label classifier + deterministic comparison rules	Explainable verdicts, no hallucinated discrepancies
Document reading	<code>pdfplumber</code> , <code>openpyxl</code> , <code>unpdf</code>	Tables and varied page layouts across four file formats
Generative AI	Google Gemini	Reply drafting only, on explicit user action
Summarisation	Local extractive summariser	TL;DR on every email with no API key and no data leaving the server
Hosting	Vercel (web) · Railway (gateway)	Serverless web tier, long-lived container for the models

Cloud infrastructure and AI integration

Both are core to the running system rather than bolted on:

- **Deployed and publicly accessible** on Vercel's global edge network, with preview deployments per branch and environment-scoped secrets.
- **Google Cloud** provides the OAuth consent flow and the Gmail API that supplies live mail.
- **Google Gemini** writes contextual reply drafts from the email and its comparison result, server-side, only when the user turns the AI Draft switch on.
- **Container-hosted model gateway** keeps the trained models resident in memory, independently scalable from the web tier.

Beyond the brief

The brief's starting version is JSON records and plain-text attachments. The submitted system goes past that on every advanced axis the use case names:

Advanced challenge	What we built
PDF and Word attachments	All four formats read, including tables and multi-column layouts — 58 of the dataset's 250 attachments are not plain text
Scanned documents	Detected and escalated as <code>unreadable</code> with the evidence, rather than silently returning empty fields
Messier inputs	Synonymous field labels resolved; misleading filenames caught by reading contents, not names — a file named <code>..._BL.txt</code> containing a packing list is reported as the wrong document type
Reliability and human review	Four distinct escalation reasons, each with source evidence; a Resolve action that moves a case out of the queue and can be reversed
<i>Not required by the brief</i>	Runs on a real Gmail inbox, not only the supplied dataset; AI reply drafting; delivery tracking with notifications

Engineering quality

- **25 automated tests** on the classifier, all passing.
- **Clean type-check, lint and production build** verified before every push.
- **Runtime response validation** — every gateway response is checked against the expected shape before it reaches the UI, so a backend change cannot corrupt the display.
- **Roughly 7,700 lines of TypeScript and 2,400 lines of Python**, documented with a README, a local setup guide and a deployment runbook.
- **Security** — attachments are streamed through and never written to disk, and are served under a sandbox Content-Security-Policy so a hostile attachment cannot execute against the application origin.

5. Challenges faced

Challenge	How it was resolved
Distinguishing a real discrepancy from a reading failure	The hardest problem in the brief. An unreadable document produces empty fields, which naively compares as “every field mismatched”. We made readability an explicit gate <i>before</i> comparison: if either document cannot be used, the run stops and escalates rather than producing a misleading report.
Misleading filenames	A file named <code>email_504_BL.txt</code> contains a packing list. Classifying documents by filename is fast and wrong. Document type is determined by reading contents, and the UI states what each file was actually read as, so an operator is never left guessing why a pair was flagged.
Gmail API quota exhaustion	Reply-delivery tracking polled the API every two seconds for up to fifteen minutes, re-fetching the same messages and exhausting quota. Resolved by splitting the endpoint so a poll lists message IDs only and fetches a message solely when a genuinely new one appears — a sevenfold reduction in quota units.
Confirming a reply was actually sent	Browser security severs the handle to a popup window, and Gmail's search index lags several seconds behind a send. Both made naive detection report false failures. Resolved by watching Sent-label message IDs against a snapshot taken before composing.
Merging three parallel work streams	Three team members built the classifier, the comparator and the web application on separate branches with independently trained models. The models load by their original module paths, so the gateway reconciles them at import time rather than restructuring either project and breaking its own artefacts.
Accuracy on real mail versus the dataset	A model scoring 520/520 on templated data is out of its depth on an arbitrary inbox. Rather than hide this, the confidence gate reports <code>GENERAL</code> below 0.55 and preserves the original prediction, and the limitation is documented rather than glossed over.

6. Known limitations

Stated deliberately. A system an operations team is asked to trust should be explicit about where it stops being reliable.

- **Fitted to the supplied dataset.** It was generated from a small set of templates, so a perfect score demonstrates the brief is satisfied on the data provided, not that the model reads arbitrary mail. On a personal inbox, most messages land in General or Spam.
- **Scanned documents are escalated, not read.** OCR would recover them; it is identified as the highest-value next step, not built.
- **Seven fields are compared** — exactly those the brief mandates. A difference in vessel or HS code is displayed but not flagged as a defect.
- **The comparison logic has a second implementation** in TypeScript, used as the offline fallback. The two are kept in step manually; consolidating them is scheduled work.
- **Gmail category counts cover the newest 1,000 messages** and display “N+” beyond that, because classifying an entire mailbox would mean downloading it.
- **Google sign-in is limited to accounts on the OAuth test-user list**, since Gmail scopes are restricted and full verification requires a paid third-party security assessment. The demo account exists so no reviewer has to be added.

7. Future roadmap

Horizon	Work	Value
Immediate	OCR for scanned documents, via a vision model or a dedicated OCR stage inserted at the readability gate	Converts the largest remaining escalation class into automated comparisons
Immediate	Consolidate the two comparison implementations behind one service contract	Removes the only place where logic is maintained twice
Near term	Continuous learning from operator corrections: every Resolve becomes a labelled example, weighted above synthetic training data	Directly addresses the fitted-to-templates limitation
Near term	Configurable field sets and per-carrier comparison rules	Different trade lanes and carriers check different fields
Medium term	Shared team state: assignment, audit trail, and an operations dashboard over escalation reasons and turnaround time	Moves from a single-operator tool to a team system
Medium term	Direct carrier and forwarder integrations, replacing mail parsing where an API exists	Removes the email round-trip entirely for partners who support it
Production	Google OAuth verification and CASA assessment; per-tenant isolation and retention policy	Required before any real shipping operation could adopt it

Scalability position today

The architecture already supports the first order of magnitude. The web tier is serverless and scales per request; the model gateway is a stateless container that scales horizontally behind a load balancer; the mail source is paged and quota-aware rather than bulk-loading; and no component holds per-user state that would prevent adding instances. The work above is about capability and trust, not about surviving load.

8. Evidence against the judging criteria

Final round weighting, with where each item is demonstrated.

Criterion	Pts	Evidence
End-to-End Functionality	25	Deployed and publicly reachable. Inbox → classification → extraction → comparison → verdict → operator action → reply, with no manual step in between. Runs on the supplied dataset and on a live Gmail mailbox.
Architecture & Scalability	15	Two independently deployable services split along their runtime needs; stateless model gateway; paged, quota-aware mail access; designed fallback when the gateway is down.
Technology Integration	15	Gmail API, Google OAuth, Gemini, two trained scikit-learn models, four document formats, and two cloud platforms, integrated into one product surface.
Engineering Quality & Robustness	15	25 passing tests; clean type-check, lint and build; runtime response validation; graceful degradation on every external dependency; documented escalation instead of silent failure.
Solution Effectiveness & User Value	10	519/520 verdicts exactly right, zero false alarms across 454 clean pairs. An operator sees the answer and the reason without opening either document.
User Experience & Differentiation	10	A working mail client, not a results table: three panes, inline previews, side-by-side field comparison, resizable split, light and dark themes, AI reply drafting with delivery tracking, and a no-credentials demo login.
Impact & Future Potential	10	Addresses a repetitive, error-prone task with a direct cost. Roadmap converts operator corrections into training data, so accuracy improves with use.

IN ONE LINE

WayBoxAI reads the inbox, checks the documents, and knows when to ask a person — with the numbers to show it, and the honesty to say where it stops.

All figures in this report were reproduced against the organisers' `ground_truth.json` on 21 September 2026 by running the deployed pipeline over all 520 emails.